



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Developer Guide

Java & Android

Version 1.30

March 2026

CONFIDENTIAL

Table of Contents

Revision History	4
Abbreviations	6
1 Introduction	7
1.1 Document Purpose	7
2 Development Prerequisites & Environment Setup	8
2.1 Java Environment Prerequisites	8
2.2 Java Environment Setup	8
2.3 Android Environment Prerequisites	8
2.4 Android Environment Setup	8
2.4.1 Mandatory Permissions	8
2.4.2 Permission Required to Use NFC Interface	8
2.4.3 Permission Required to Use Bluetooth Readers	8
2.4.4 Setting Required to Use USB Connected Readers	9
3 Developing Your First Program	10
3.1 Step 1: Initialize the Toolkit	10
3.2 Step 2: List Connected Readers	10
3.3 Step 3: Connect to a Card Reader	10
3.4 Step 4: Read Public Data	11
3.5 Step 5: Disconnect	11
4 Development Best Practices	12
4.1 Android Development Recommendations	12
4.1.1 Asynchronous Tasks	12
5 Toolkit API Reference	13
5.1 Class: Toolkit	13
5.1.1 Constructor: Toolkit	13
5.1.2 Method: getToolkitVersion	13
5.1.3 Method: setNfcMode	14
5.1.4 Method: cleanup	14
5.1.5 Method: listReaders	14
5.1.6 Method: getReaderWithEmiratesID	15
5.1.7 Method: prepareRequest	15
5.1.8 Method: registerDevice	15
5.1.9 Method: getDeviceId	16
5.1.10 Method: getDataProtectionKey	16
5.1.11 Method: parseMRZ	16
5.1.12 Method: getConfigCertificateExpiryDate	17
5.1.13 Method: getLicenseExpiryDate	17
5.2 Class: CardReader	17
5.2.1 Method: connect	17
5.2.2 Method: disconnect	17
5.2.3 Method: getReaderName	18
5.2.4 Method: getReaderSerialNumber	18
5.2.5 Method: getCardVersion	18
5.2.6 Method: prepareRequest (CardReader)	18
5.2.7 Method: setNfcAuthenticationParameters (Card Attributes)	19

5.2.8 Method: setNfcAuthenticationParameters (MRZ)	19
5.2.9 Method: checkCardStatus	19
5.2.10 Method: getCardSerialNumber	20
5.2.11 Method: readPublicData	20
5.2.12 Method: readPublicDataEF	21
5.2.13 Method: parseEFData	21
5.2.14 Method: getPkiCertificates	22
5.2.15 Method: authenticatePki	22
5.2.16 Method: signData	22
5.2.17 Method: signChallenge	23
5.2.18 Method: getFingerData	23
5.2.19 Method: authenticateBiometricOnServer	24
5.2.20 Method: authenticateCardAndBiometric	24
5.2.21 Method: resetPin	25
5.2.22 Method: resetPINWithoutAuthenticateBiometric	25
5.2.23 Method: unblockPin	25
5.2.24 Method: readFamilyBook	26
5.2.25 Method: updateData	26
5.2.26 Method: readData	27
5.2.27 Method: verifySignature	27
5.3 Class: SignatureValidator	28
5.3.1 Constructor: SignatureValidator	28
5.3.2 Method: ValidateToolkitResponse	28
6 Advanced Digital Signature API Reference	29
6.1 Method: padesSign	29
6.2 Method: padesVerify	29
6.3 Method: xadesSign	30
6.4 Method: xadesVerify (Enveloped/Enveloping)	30
6.5 Method: xadesVerify (Detached)	30
6.6 Method: cadesSign	31
6.7 Method: cadesVerify	31

Revision History

Version	Date	Description of Changes
1.0	22nd Feb, 2017	Beta Release 1
1.1	17th Apr, 2017	Beta Release 2
1.2	09th Aug, 2017	Beta Release 3
1.3	10th Dec, 2017	Release Version 1.0.0
1.4	30th Mar, 2018	Release Version 1.0.3
1.5	28th May, 2018	Release Version 1.0.5
1.6	12th Jun, 2018	Release Version 1.0.6
1.7	02nd Aug, 2018	Release Version 1.0.8
1.8	19th Sep, 2018	Release Version 1.0.8
1.9	12th Dec, 2018	Release Version 1.0.9
1.10	16th Feb, 2019	Release Version 1.0.13
1.11	19th Mar, 2019	Release Version 1.1.0
1.12	03rd Apr, 2019	Release Version 1.2.0
1.13	13th May, 2019	Release Version 1.3.0
1.14	11th Jun, 2019	Release Version 1.4.0
1.15	17th Jun, 2019	Release Version 1.4.1
1.16	31st Jul, 2019	Release Version 1.4.3; Included APIs for Card Update functionality.
1.17	04th Mar, 2020	Release Version 1.4.5
1.18	01st Apr, 2020	Release Version 1.4.5-r2
1.19	05th Apr, 2020	Release Version 1.4.6
1.20	02nd Nov, 2020	Release Version 1.4.8
1.21	24th May, 2021	Release Version 1.4.9
1.22	10th Oct, 2021	Release Version 2.0.1-r2
1.23	15th Nov, 2021	Release Version 2.0.3-r1
1.24	10th Mar, 2022	Release Version 2.0.4
1.25	12th Jun, 2022	Release Version 2.0.5-r1

Version	Date	Description of Changes
1.26	20th Dec, 2023	Release Version 3.0.0
1.27	14th Mar, 2024	Release Version 3.0.1-r1
1.28	07th May, 2024	Release Version 3.0.1-r5
1.29	21st May, 2025	Release Version 3.0.3-r1
1.30	Mar 2026	Toolkit v3.0.5. Improved code samples.

Abbreviations

Abbreviation	Description
AAR	Android Archive
CA	Certificate Authority
CAdES	CMS Advanced Electronic Signatures
CMS	Cryptographic Message Syntax
EF	Elementary File
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
JDK	Java Development Kit
JAR	Java Archive
JNI	Java Native Interface
JRE	Java Runtime Environment
MRZ	Machine Readable Zone
NFC	Near Field Communication
PAdES	PDF Advanced Electronic Signatures
PDF	Portable Document Format
SP	Service Provider
VG	Validation Gateway
XAdES	XML Advanced Electronic Signatures
XML	Extensible Markup Language

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Document Purpose

This document is a developer's guide to support the development of Java and Android applications that integrate the ICP ID Card Toolkit SDK.

2. Development Prerequisites & Environment Setup

2.1. Java Environment Prerequisites

- JDK/JRE version 1.6 (Update 43) or higher
- ICP ID Card Toolkit SDK for Java

2.2. Java Environment Setup

1. Add `EIDAToolkit.jar` to the classpath.
2. Add the native libraries used by the Toolkit to the Java library path.

2.3. Android Environment Prerequisites

- JDK/JRE version 1.8 or higher
- ID Card Toolkit SDK Android binaries
- Minimum API Level 15
- Android Studio

2.4. Android Environment Setup

1. Create a project in Android Studio.
2. Set the minimum Android SDK level required by the application. The minimum supported API level is 15.
3. Add `EIDAToolkit.aar` to the project as a module dependency. All required native libraries are included in the archive.
4. Add the device plugin library as a module dependency to the project.
5. Create the `EIDAToolkit` folder in the root of the external storage.
6. Copy the configuration files into the `EIDAToolkit` folder.
7. Set up the permissions described below.

2.4.1. Mandatory Permissions

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
```

2.4.2. Permission Required to Use NFC Interface

```
<uses-permission android:name="android.permission.NFC" />
```

2.4.3. Permission Required to Use Bluetooth Readers

```
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.BLUETOOTH" />
```

2.4.4. Setting Required to Use USB Connected Readers

```
<uses-feature android:name="android.hardware.usb.host" />
```

3. Developing Your First Program

The following example shows the sequence of API calls required to develop an application using the Toolkit SDK for Java/Android. The example illustrates how to use the Read Public Data service. To use other services, follow a similar approach.

Refer to the Toolkit API Reference (Section 5) for details of all available services.

3.1. Step 1: Initialize the Toolkit

Toolkit initialization is performed by the constructor of the `Toolkit` class.

Java:

```
// Create a new object of Toolkit
Toolkit toolkit = new Toolkit(true, configParams);
```

Android: The Android constructor takes an additional `context` parameter. Initialization will fail if the context is not provided.

```
Context context = getApplicationContext();

// Create a new object of Toolkit
Toolkit toolkit = new Toolkit(true, configParams, context);
```

3.2. Step 2: List Connected Readers

Retrieve a list of connected smartcard readers using the `listReaders` method. This returns an array of `CardReader` objects. If no compatible readers are found, an empty array is returned.

```
CardReader[] readers = toolkit.listReaders();
```

3.3. Step 3: Connect to a Card Reader

Connect to a card reader by calling the `connect` method on a `CardReader` object.

```
// Connect to the first card reader returned by listReaders
CardReader cardReader = readers[0];
cardReader.connect();
```

Note: To directly access a card reader with an Emirates ID card inserted, use the `getReaderWithEmiratesID` method:

```
CardReader cardReader = toolkit.getReaderWithEmiratesID();
cardReader.connect();
```

3.4. Step 4: Read Public Data

Call `readPublicData` to retrieve public data from the connected card. Generate a Request ID before each service call. Refer to the "ID Card Toolkit Programmer's Reference" for guidelines.

```
CardPublicData cardPublicData = cardReader.readPublicData(  
    requestID,  
    readNonModifiableData,  
    readModifiableData,  
    readPhotography,  
    readSignatureImage,  
    readAddress);
```

If successful, the method returns a `CardPublicData` object. The `xmlString` property contains the digitally signed Toolkit Response XML. Applications should validate the response digital signature and Request ID. Refer to the sample program provided with the SDK.

3.5. Step 5: Disconnect

Disconnect from the card when done.

```
cardReader.disconnect();
```

4. Development Best Practices

4.1. Android Development Recommendations

4.1.1. Asynchronous Tasks

On the Android platform, SDK operations should be implemented on a background thread to prevent impact on application responsiveness. This can be done using RxAndroid thread scheduler, custom threads, `AsyncTask`, `Handler`, or Kotlin Coroutines.

5. Toolkit API Reference

5.1. Class: Toolkit

This class provides methods required to initialize the Toolkit and get the connected smartcard readers.

5.1.1. Constructor: Toolkit

Initializes the Toolkit context and prepares the SDK for use.

Syntax

Java:

```
public Toolkit(  
    boolean inProcessMode,  
    String configParams  
) throws ToolkitException;
```

Android:

```
public Toolkit(  
    boolean inProcessMode,  
    String configParams,  
    Object context  
) throws ToolkitException;
```

Parameter	Type	Description
inProcessMode	boolean	When <code>true</code> , the Toolkit runs in-process. When <code>false</code> , API requests are forwarded to the Toolkit Agent. Agent mode is not supported on Android.
configParams	String	Configuration parameters as a string. If a file is used, read its content and pass it as a string. Refer to the Programmer's Reference for details.
context	Object	Android application context (Android only). Obtained via <code>getApplicationContext()</code> .

Throws: `ToolkitException` on error.

5.1.2. Method: getToolkitVersion

Returns the version of the Toolkit being used by the application.

Syntax

```
public String getToolkitVersion() throws ToolkitException;
```

Parameters: None

Return: String – Toolkit version in major.minor.patch format. **Throws:** ToolkitException on error.

5.1.3. Method: setNfcMode

Sets the NFC tag object for the Toolkit. This method must be called every time before calling connect, listReaders, OR getReaderWithEmiratesID. Refer to the Android sample for usage details.

Note: This method is used only with the Android SDK along with the NFC Reader plugin.

Syntax

```
public void setNfcMode(Object tag) throws ToolkitException;
```

Parameter	Type	Description
tag	Object	NFC tag object. A new tag object is created every time a tag is discovered (card tapped). The tag is invalidated when the card is removed, goes out of range, or when disconnect is called.

Return: void. **Throws:** ToolkitException on error.

5.1.4. Method: cleanup

Releases resources used by the Toolkit and closes the service context. In in-process mode, cleanup is performed within the library. In agent mode, a cleanup request is sent to the agent.

Note: Agent mode is not supported on Android.

Syntax

```
public void cleanup() throws ToolkitException;
```

Parameters: None

Return: void. **Throws:** ToolkitException on error.

5.1.5. Method: listReaders

Lists all compatible smartcard readers connected to the system. On Android, this lists readers from registered plugins.

Syntax

```
public CardReader[] listReaders() throws ToolkitException;
```

Parameters: None

Return: CardReader[] – array of connected smartcard readers. Returns an empty array if none found. **Throws:** ToolkitException on error.

5.1.6. Method: getReaderWithEmiratesID

Checks all smartcard readers and returns the first reader with an Emirates ID Card inserted.

Syntax

```
public CardReader getReaderWithEmiratesID() throws ToolkitException;
```

Parameters: None

Return: CardReader – the reader with an Emirates ID Card inserted. **Throws:** ToolkitException on error.

5.1.7. Method: prepareRequest

Establishes a secure context for execution of a Toolkit service. The Request Handle returned in the Toolkit Response XML is used to encode sensitive information (User ID, Password, PIN) to protect against sniffing and replay attacks.

Syntax

```
public String prepareRequest(String requestId) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier. Refer to the Programmer's Reference for guidelines.

Return: String – 8-byte request handle in Base64 format. **Throws:** ToolkitException on error.

5.1.8. Method: registerDevice

Registers a device with the Validation Gateway (VG) against the Service Provider (SP) license. prepareRequest must be called successfully before this method.

Syntax

```
public RegisterDeviceResponse registerDevice(  
    String encodedUserId,  
    String encodedPassword,  
    int deviceReferenceId  
) throws ToolkitException;
```

Parameter	Type	Description
encodedUserId	String	SP User ID, encrypted and Base64-encoded per the Programmer's Reference.
encodedPassword	String	Password, encrypted and Base64-encoded per the Programmer's Reference.
deviceReferenceId	int	Unique device ID generated by the SP for internal device management.

Return: RegisterDeviceResponse – contains deviceRegistrationID. Extends ToolkitResponse; the xmlString property returns the XML response for signature validation. **Throws:** ToolkitException on error.

5.1.9. Method: getDeviceId

Retrieves the unique device identifier, required to deregister a device through the ICP Web Service REST API.

Syntax

```
public String getDeviceId() throws ToolkitException;
```

Parameters: None

Return: String – the device identifier. **Throws:** ToolkitException on error.

5.1.10. Method: getDataProtectionKey

Retrieves the VG public key used for data protection. This key is used to encode sensitive parameters (User ID, Password, PIN).

Syntax

```
public DataProtectionKey getDataProtectionKey() throws ToolkitException;
```

Parameters: None

Return: DataProtectionKey – contains the public key. Key methods:

- getPublicKey() – returns the public key as byte[]
- getModulus() – returns the modulus as String
- getExponent() – returns the exponent as String

Refer to the Programmer's Reference for encoding procedures. **Throws:** ToolkitException on error.

5.1.11. Method: parseMRZ

Parses a Machine Readable Zone (MRZ) string and retrieves the corresponding attributes.

Syntax

```
public MRZData parseMRZ(String mrz) throws ToolkitException;
```

Parameter	Type	Description
mrz	String	The MRZ string to parse.

Return: MRZData – parsed attributes accessible through getter methods:

- getCardNumber(), getIDNumber(), getDocumentType(), getIssuedCountry()
- getDateOfBirth(), getGender(), getNationality(), getFullName(), getCardExpiryDate()

Throws: ToolkitException on error.

5.1.12. Method: getConfigCertificateExpiryDate

Returns the expiry dates of certificates corresponding to the Toolkit SDK configuration files.

Syntax

```
public ConfigExpiryDate getConfigCertificateExpiryDate() throws ToolkitException;
```

Parameters: None

Return: ConfigExpiryDate – certificate expiry dates for each configuration file, accessible through getter methods. **Throws:** ToolkitException on error.

5.1.13. Method: getLicenseExpiryDate

Returns the expiry date of the Toolkit SDK license issued to the Service Provider.

Syntax

```
public String getLicenseExpiryDate() throws ToolkitException;
```

Parameters: None

Return: String – the license expiry date. **Throws:** ToolkitException on error.

5.2. Class: CardReader

This class provides all methods required to perform operations with the Emirates ID Card after a successful connect operation.

5.2.1. Method: connect

Establishes a connection to the ID card in the reader. This method must be called each time an ID card is presented or when the reader is reconnected. Once connected, multiple Toolkit services can be invoked. Reusing the connection for multiple services within a single ID card session improves performance.

Syntax

```
public void connect() throws ToolkitException;
```

Parameters: None

Return: void. **Throws:** ToolkitException on error.

5.2.2. Method: disconnect

Disconnects from the connected reader. The ID card can be safely removed after disconnect.

Syntax

```
public void disconnect() throws ToolkitException;
```

Parameters: None

Return: void. **Throws:** ToolkitException on error.

5.2.3. Method: `getReaderName`

Retrieves the name of the connected smartcard reader.

Syntax

```
public String getReaderName() throws ToolkitException;
```

Parameters: None

Return: String – the reader name. **Throws:** ToolkitException on error.

5.2.4. Method: `getReaderSerialNumber`

Retrieves the serial number of the connected smartcard reader.

Note: This method is used only with the Windows (desktop Java) SDK.

Syntax

```
public String getReaderSerialNumber() throws ToolkitException;
```

Parameters: None

Return: String – the reader serial number, or null if not available. **Throws:** ToolkitException on error.

5.2.5. Method: `getCardVersion`

Retrieves the version number of the connected Emirates ID Card.

Syntax

```
public int getCardVersion() throws ToolkitException;
```

Parameters: None

Return: int – the card version number. **Throws:** ToolkitException on error.

Value	Card Version
2	V2
3	V3
4	V4
5	V5

V5 is the latest generation card with enhanced security. Full read, PKI, and biometric operations are supported.

5.2.6. Method: `prepareRequest (CardReader)`

Establishes a secure context for execution of a Toolkit service on the connected card.

Syntax

```
public String prepareRequest(String requestId) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.

Return: String – 8-byte request handle in Base64 format. **Throws:** ToolkitException on error.

5.2.7. Method: setNfcAuthenticationParameters (Card Attributes)

Sets NFC authentication parameters using specific card attributes printed on the card surface.

Syntax

```
public void setNfcAuthenticationParameters(  
    String cardNumber,  
    String dateOfBirth,  
    String expiryDate  
) throws ToolkitException;
```

Parameter	Type	Description
cardNumber	String	Card number of the Emirates ID Card holder.
dateOfBirth	String	Date of birth in YYMMDD format. Example: 12th April 1956 = 560412.
expiryDate	String	Card expiry date in YYMMDD format. Example: 16th November 2027 = 271116.

Return: void. **Throws:** ToolkitException on error.

5.2.8. Method: setNfcAuthenticationParameters (MRZ)

Sets NFC authentication parameters using the MRZ string from the back of the ID card. How to acquire the MRZ string is outside the scope of the Toolkit (e.g., OCR library or MRZ scanner).

Syntax

```
public void setNfcAuthenticationParameters(  
    String mrzData  
) throws ToolkitException;
```

Parameter	Type	Description
mrzData	String	Scanned Machine Readable Zone (MRZ) data from the back of the ID Card.

Return: void. **Throws:** ToolkitException on error.

5.2.9. Method: checkCardStatus

Validates the Emirates ID Card with the ICP Validation Gateway (VG) and returns the card status.

Syntax

```
public ToolkitResponse checkCardStatus(String requestId) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.

Return: ToolkitResponse – contains the card status. The `xmlString` property returns the XML response for signature validation. **Throws:** ToolkitException on error.

5.2.10. Method: getCardSerialNumber

Retrieves the serial number of the connected Emirates ID Card.

Syntax

```
public String getCardSerialNumber() throws ToolkitException;
```

Parameters: None

Return: String – the card serial number. **Throws:** ToolkitException on error.

5.2.11. Method: readPublicData

Retrieves data stored on the public areas of the Emirates ID Card. Applications can use flags to select which data categories are returned, improving performance. The data is digitally signed for integrity and verified within the Toolkit.

Syntax

```
public CardPublicData readPublicData(  
    String requestId,  
    boolean readNonModifiableData,  
    boolean readModifiableData,  
    boolean readPhotography,  
    boolean readSignatureImage,  
    boolean readAddress  
) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.
readNonModifiableData	boolean	true to read non-modifiable data (name, nationality, etc.).
readModifiableData	boolean	true to read modifiable data (occupation, employer, etc.).
readPhotography	boolean	true to read the card holder photograph.
readSignatureImage	boolean	true to read the handwritten signature image.

Parameter	Type	Description
readAddress	boolean	true to read home and work address data.

Return: CardPublicData – contains the requested public data. Key attributes: idNumber, cardNumber, cardHolderPhoto, holderSignatureImage, ModifiablePublicData, NonModifiablePublicData, HomeAddress, WorkAddress. The xmlString property returns the digitally signed XML response. **Throws:** ToolkitException on error.

5.2.12. Method: readPublicDataEF

Retrieves Elementary File data from the public areas of the ID card.

Syntax

```
public byte[] readPublicDataEF(  
    int publicDataEFTType,  
    boolean validateSignature  
) throws ToolkitException;
```

Parameter	Type	Description
publicDataEFTType	int	Elementary File identifier (see table below).
validateSignature	boolean	true to validate the digital signature of the EF data.

Value	Constant	EF Data
1	IDN_CN	Identification and Card Number
2	ROOT_CERTIFICATE	Root Certificate
3	NON_MODIFIABLE_DATA	Non-Modifiable Public Data
4	MODIFIABLE_DATA	Modifiable Public Data
5	PHOTOGRAPHY	Photography
6	SIGNATURE_IMAGE	Handwritten Signature Image
7	HOME_ADDRESS	Home Address
8	WORK_ADDRESS	Work Address

Return: byte[] – the Elementary File data. **Throws:** ToolkitException on error.

5.2.13. Method: parseEFData

Parses Elementary File data into a JSON string. This is a static method and can be called without a CardReader object.

Syntax

```
public static String parseEFData(byte[] efData) throws ToolkitException;
```

Parameter	Type	Description
efData	byte[]	Elementary File data bytes to parse (obtained from readPublicDataEF).

Return: String – parsed Elementary File data as a JSON string. **Throws:** ToolkitException on error.

5.2.14. Method: getPkiCertificates

Reads the authentication and signing certificates from the Emirates ID Card. prepareRequest must be called successfully before this method.

Syntax

```
public CardCertificates getPkiCertificates(  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: CardCertificates – contains authentication and signing certificates. The xmlString property returns the XML response. **Throws:** ToolkitException on error. The getAttemptsLeft() method from ToolkitException returns remaining PIN attempts.

5.2.15. Method: authenticatePki

Authenticates the ID Card PIN and verifies the revocation status of the authentication certificate. prepareRequest must be called successfully before this method.

Syntax

```
public ToolkitResponse authenticatePki(  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: ToolkitResponse – contains the PKI authentication status. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining PIN attempts.

5.2.16. Method: signData

Digitally signs plain data or a hash using the signature certificate of the ID card. prepareRequest must be called successfully before this method.

Syntax

```
public SignatureResponse signData(  
    byte[] input,  
    boolean isInputHashed,  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
input	byte[]	Data or hash to digitally sign.
isInputHashed	boolean	true if the input is already hashed; false if plain data (Toolkit applies hash before signing).
encodedPin	String	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: SignatureResponse – contains the signature via getSignature() and the signing status. The xmlString property returns the XML response. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining PIN attempts.

5.2.17. Method: signChallenge

Digitally signs challenge data using the authentication certificate of the ID card. Used to authenticate users through challenge signing and verification. prepareRequest must be called successfully before this method.

Syntax

```
public SignatureResponse signChallenge(  
    byte[] input,  
    boolean isInputHashed,  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
input	byte[]	Challenge bytes to digitally sign.
isInputHashed	boolean	true if the input is already hashed; false if plain challenge data.
encodedPin	String	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: SignatureResponse – contains the signature and signing status. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining PIN attempts.

5.2.18. Method: getFingerData

Returns finger index values and biometric reference identifiers of the fingerprints stored on the Emirates ID Card.

Syntax

```
public FingerData[] getFingerData() throws ToolkitException;
```

Parameters: None

Return: FingerData[] – array of finger data objects containing biometric reference identifiers and finger index values. Refer to the Programmer's Reference for finger index values. **Throws:** ToolkitException on error.

5.2.19. Method: authenticateBiometricOnServer

Performs biometric authentication with the VG service against a captured finger image.

Syntax

```
public ToolkitResponse authenticateBiometricOnServer(  
    String requestId,  
    FingerIndex fingerIndex,  
    int sensorTimeout  
) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.
fingerIndex	FingerIndex	Finger index from a previous getFingerData call.
sensorTimeout	int	Timeout in seconds for the fingerprint sensor.

Return: ToolkitResponse – contains the biometric authentication status. **Throws:** ToolkitException on error.

5.2.20. Method: authenticateCardAndBiometric

Validates the Emirates ID Card and performs biometric authentication with the VG service against a captured finger image.

Syntax

```
public ToolkitResponse authenticateCardAndBiometric(  
    String requestId,  
    FingerIndex fingerIndex,  
    int sensorTimeout  
) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.
fingerIndex	FingerIndex	Finger index from a previous getFingerData call.
sensorTimeout	int	Timeout in seconds for the fingerprint sensor.

Return: ToolkitResponse – contains the biometric authentication status. **Throws:** ToolkitException on error.

5.2.21. Method: resetPin

Resets the ID Card PIN using the VG service. Involves biometric authentication. prepareRequest must be called successfully before this method.

Syntax

```
public ToolkitResponse resetPin(  
    String encodedPin,  
    FingerData fingerData,  
    int sensorTimeout  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
fingerData	FingerData	Finger data object from a previous getFingerData call.
sensorTimeout	int	Timeout in seconds for the fingerprint sensor.

Return: ToolkitResponse – contains the reset status via responseStatus. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining attempts.

5.2.22. Method: resetPINWithoutAuthenticateBiometric

Resets the ID Card PIN using the VG service without biometric authentication. prepareRequest must be called successfully before this method.

Syntax

```
public ToolkitResponse resetPINWithoutAuthenticateBiometric(  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	New PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: ToolkitResponse – contains the reset status. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining attempts.

5.2.23. Method: unblockPin

Unblocks the ID Card PIN after the card has been blocked due to multiple unsuccessful PIN attempts. Involves biometric authentication. prepareRequest must be called successfully before this method.

Syntax

```
public ToolkitResponse unblockPin(  
    String encodedPin,  
    FingerData fingerData,  
    int sensorTimeout  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
fingerData	FingerData	Finger data object from a previous getFingerData call.
sensorTimeout	int	Timeout in seconds for the fingerprint sensor.

Return: ToolkitResponse – contains the unblock status. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining attempts.

5.2.24. Method: readFamilyBook

Reads the family book data from the Emirates ID Card. Family book data is protected and requires PIN authentication. prepareRequest must be called successfully before this method.

Syntax

```
public CardFamilyBookData readFamilyBook(  
    String encodedPin  
) throws ToolkitException;
```

Parameter	Type	Description
encodedPin	String	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: CardFamilyBookData – contains HeadOfFamily, wives[], and children[]. The xmlString property returns the XML response. **Throws:** ToolkitException on error. The getAttemptsLeft() method returns remaining PIN attempts.

5.2.25. Method: updateData

Updates data on the Emirates ID Card. The update can be for existing data (modifiable data, address, family book) or for new containers.

Syntax

```
public ToolkitResponse updateData(  
    String requestId,  
    int fileType  
) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.
fileType	int	Identifier of the file/container to update (see table below).

Value	File / Container
0	HEALTH_INSURANCE
1	HEALTH_INSURANCE_PROTECTED
2	MOD_DATA
3	HOMEADDRESS
4	WORKADDRESS
5	HEAD_OF_FAMILY
6–9	WIFE1 – WIFE4
10–29	CHILD1 – CHILD20

Return: ToolkitResponse – contains the update status. **Throws:** ToolkitException on error.

5.2.26. Method: readData

Reads updated container data from the Emirates ID Card. The data is digitally signed for integrity and verified within the Toolkit.

Syntax

```
public ToolkitResponse readData(
    String requestId,
    int fileType
) throws ToolkitException;
```

Parameter	Type	Description
requestId	String	Application-generated request identifier.
fileType	int	Identifier of the container to read (0 = HEALTH_INSURANCE, 1 = HEALTH_INSURANCE_PROTECTED).

Return: ToolkitResponse – contains the requested data. **Throws:** ToolkitException on error.

5.2.27. Method: verifySignature

Verifies a digital signature with the corresponding certificate provided by the application.

Syntax

```
public void verifySignature(  
    byte[] input,  
    boolean isInputHash,  
    byte[] signature,  
    byte[] certificate  
) throws ToolkitException;
```

Parameter	Type	Description
input	byte[]	Data or hash corresponding to the digital signature.
isInputHash	boolean	true if hash data is provided; false if plain data.
signature	byte[]	The digital signature to verify.
certificate	byte[]	Certificate data for signature verification.

Return: void. **Throws:** ToolkitException if verification fails or on error.

5.3. Class: SignatureValidator

This class provides a method to validate Toolkit XML Response received from various API operations.

5.3.1. Constructor: SignatureValidator

Sets the attributes required to verify the Toolkit XML Response.

Syntax

```
public SignatureValidator(byte[] cert, byte[] certChain);
```

Parameter	Type	Description
cert	byte[]	Certificate data for response verification.
certChain	byte[]	Issuer certificate chain data. Pass null to skip chain validation.

5.3.2. Method: ValidateToolkitResponse

Validates a Toolkit XML response signature.

Syntax

```
public void ValidateToolkitResponse(String VGResponse) throws ToolkitException;
```

Parameter	Type	Description
VGResponse	String	Toolkit Response XML string.

Return: void. On success, validated attributes can be retrieved through getter methods on the SignatureValidator instance. **Throws:** ToolkitException on error.

6. Advanced Digital Signature API Reference

The Toolkit provides APIs for creating and verifying advanced electronic signatures conforming to PAdES, XAdES, and CAdES standards.

6.1. Method: `padesSign`

Digitally signs a PDF document in conformance with PAdES (PDF Advanced Electronic Signatures) per ETSI EN 319 142. `prepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse padesSign(  
    PadesSigningContext signingContext,  
    String pdfFilePath,  
    String signedPdfFilePath  
) throws ToolkitException;
```

Parameter	Type	Description
<code>signingContext</code>	<code>PadesSigningContext</code>	PAdES-specific signature properties including PIN, signature level, visible signature appearance, and TSA/OCSP URLs. Refer to the Programmer's Reference for details.
<code>pdfFilePath</code>	<code>String</code>	Path of the input PDF document.
<code>signedPdfFilePath</code>	<code>String</code>	Path to store the signed PDF document.

Return: `ToolkitResponse` – contains the signing status via `responseStatus`. **Throws:** `ToolkitException` on error.

6.2. Method: `padesVerify`

Verifies a PAdES-signed PDF document.

Syntax

```
public String padesVerify(  
    VerificationContext verificationContext,  
    String signedPdfFilePath  
) throws ToolkitException;
```

Parameter	Type	Description
<code>verificationContext</code>	<code>VerificationContext</code>	Verification context properties. Refer to the Programmer's Reference for details.
<code>signedPdfFilePath</code>	<code>String</code>	Path of the signed PDF document.

Return: `String` – verification report in XML format. **Throws:** `ToolkitException` on error.

6.3. Method: xadesSign

Digitally signs an XML document in conformance with XAdES (XML Advanced Electronic Signatures) per ETSI EN 319 132. prepareRequest must be called successfully before this method.

Syntax

```
public ToolkitResponse xadesSign(  
    SigningContext signingContext,  
    String xmlFilePath,  
    String signedXmlFilePath  
) throws ToolkitException;
```

Parameter	Type	Description
signingContext	SigningContext	Signature properties including level, packaging mode, and PIN.
xmlFilePath	String	Path of the input XML document.
signedXmlFilePath	String	Path to store the signed document. Pass null for detached mode.

Return: ToolkitResponse – in detached mode, the detachedSignature property contains the signature. In enveloped/enveloping mode, the signature is embedded in the output file. **Throws:** ToolkitException ON error.

Signing Mode	signedXmlFilePath	detachedSignature
Detached	null	Signature value
Enveloped / Enveloping	Path to signed document	Empty array

6.4. Method: xadesVerify (Enveloped/Enveloping)

Verifies an XAdES-signed XML document containing an embedded signature.

Syntax

```
public String xadesVerify(  
    VerificationContext verificationContext,  
    String signedXmlFilePath  
) throws ToolkitException;
```

Parameter	Type	Description
verificationContext	VerificationContext	Verification context properties.
signedXmlFilePath	String	Path of the signed XML document.

Return: String – verification report in XML format. **Throws:** ToolkitException ON error.

6.5. Method: xadesVerify (Detached)

Verifies a detached XAdES signature against the original XML document.

Syntax

```
public String xadesVerify(  
    VerificationContext verificationContext,  
    String xmlFilePath,  
    byte[] signature  
) throws ToolkitException;
```

Parameter	Type	Description
verificationContext	VerificationContext	Verification context properties.
xmlFilePath	String	Path of the original XML document.
signature	byte[]	Detached signature bytes.

Return: String – verification report in XML format. **Throws:** ToolkitException on error.

6.6. Method: cadesSign

Digitally signs data in conformance with CADES (CMS Advanced Electronic Signatures) per ETSI EN 319 122. prepareRequest must be called successfully before this method.

Note: CADES signing works only in detached mode.

Syntax

```
public ToolkitResponse cadesSign(  
    SigningContext signingContext,  
    String inputFilePath  
) throws ToolkitException;
```

Parameter	Type	Description
signingContext	SigningContext	Signature properties. The packaging mode must be set to Detached.
inputFilePath	String	Path of the input file to sign.

Return: ToolkitResponse – the detachedSignature property contains the CADES signature. **Throws:** ToolkitException on error.

6.7. Method: cadesVerify

Verifies a CADES signature.

Syntax

```
public String cadesVerify(  
    VerificationContext verificationContext,  
    String inputFilePath,
```

```
byte[] signature  
) throws ToolkitException;
```

Parameter	Type	Description
verificationContext	VerificationContext	Verification context properties.
inputFilePath	String	Path of the original document.
signature	byte[]	CAdES signature bytes to verify.

Return: String – verification report in XML format. **Throws:** ToolkitException on error.